

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Medium (Scenario Based Assessment)

COURSE CODE: CSA09 COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)

QUESTIONS: 5 Total Marks: 100

ANNEXURE A

Scenario Based Assessment

Code of Conduct:

I **T.CHANDRA LEKHA(192472290)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1: Student Registration System

(a) Three Collection Interfaces

1. List
2. Set
3. Map

(b) Collection Structure

HashSet<Student>

|

```
-- Alice  
-- Bob  
-- Charlie
```

```
HashMap<Student, List<Course>>
```

```
Alice  -> [Java, DBMS]
```

```
Bob    -> [Python, AI]
```

```
Charlie -> [C++, OS]
```

(c) Java Program

```
import java.util.*;
```

```
public class StudentRegistration {  
    public static void main(String[] args) {
```

```
        HashSet<String> students = new HashSet<>();
```

```
        students.add("Alice");
```

```
        students.add("Bob");
```

```
        students.add("Charlie");
```

```
        HashMap<String, ArrayList<String>> map = new HashMap<>();
```

```
        map.put("Alice", new ArrayList<>(Arrays.asList("Java", "DBMS")));
```

```
        map.put("Bob", new ArrayList<>(Arrays.asList("Python", "AI"));
```

```
        map.put("Charlie", new ArrayList<>(Arrays.asList("C++", "OS"));
```

```
        Iterator<String> it = students.iterator();
```

```

while(it.hasNext()) {
    String student = it.next();
    System.out.println(student + " -> " + map.get(student));
}
}
}

```

Output

Q2: E-Commerce Cart System

(a) Three Collection Classes

1. ArrayList
2. LinkedList
3. Vector

(b) List Structure

Cart (ArrayList)

Index	Product
0	Laptop
1	Mouse
2	Keyboard
3	Pen Drive

(c) Java Program

```

import java.util.*;

public class ShoppingCart {
    public static void main(String[] args) {

```

```
ArrayList<String> cart = new ArrayList<>();
```

```
cart.add("Laptop");
```

```
cart.add("Mouse");
```

```
cart.add("Keyboard");
```

```
cart.add("Pen Drive");
```

```
cart.remove("Mouse");
```

```
Iterator<String> it = cart.iterator();
```

```
System.out.println("Cart Items:");
```

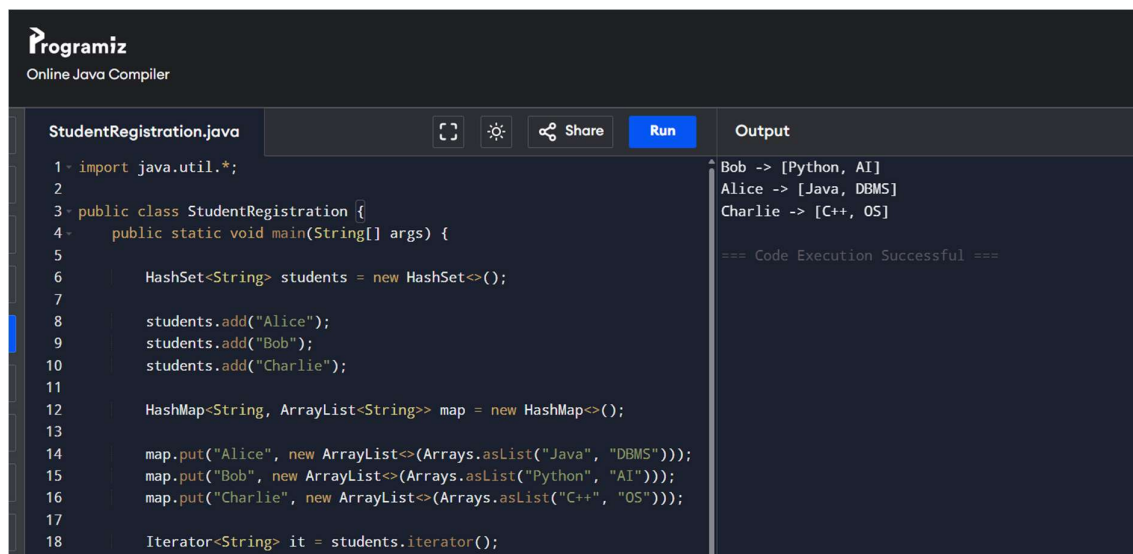
```
while(it.hasNext()) {
```

```
    System.out.println(it.next());
```

```
}
```

```
}
```

```
}
```



The screenshot displays the Programiz Online Java Compiler interface. The editor on the left contains a Java file named `StudentRegistration.java` with the following code:

```
1 import java.util.*;
2
3 public class StudentRegistration {
4     public static void main(String[] args) {
5
6         HashSet<String> students = new HashSet<>();
7
8         students.add("Alice");
9         students.add("Bob");
10        students.add("Charlie");
11
12        HashMap<String, ArrayList<String>> map = new HashMap<>();
13
14        map.put("Alice", new ArrayList<>(Arrays.asList("Java", "DBMS")));
15        map.put("Bob", new ArrayList<>(Arrays.asList("Python", "AI")));
16        map.put("Charlie", new ArrayList<>(Arrays.asList("C++", "OS")));
17
18        Iterator<String> it = students.iterator();
```

The right-hand side of the interface shows the **Output** section, which contains the following text:

```
Bob -> [Python, AI]
Alice -> [Java, DBMS]
Charlie -> [C++, OS]

=== Code Execution Successful ===
```

Q3: Library Management System

(a) Three Map Implementations

1. HashMap
2. TreeMap
3. LinkedHashMap

(b) Schema Representation

HashMap<ISBN, Book Name>

978101 -> Java Programming

978102 -> Python Basics

978103 -> Data Structures

(c) Java Program

```
import java.util.*;
```

```
public class LibrarySystem {  
    public static void main(String[] args) {
```

```
        HashMap<String, String> books = new HashMap<>();
```

```
        books.put("978101", "Java Programming");
```

```
        books.put("978102", "Python Basics");
```

```
        books.put("978103", "Data Structures");
```

```
        System.out.println("Book Found:");
```

```
        System.out.println(books.get("978102"));
```

```
        System.out.println("\nAll Books:");
```

```

        for(Map.Entry<String,String> entry : books.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}

```

Output

Programiz
Online Java Compiler

LibrarySystem.java

```

1 import java.util.*;
2
3 public class LibrarySystem {
4     public static void main(String[] args) {
5
6         HashMap<String, String> books = new HashMap<>();
7
8         books.put("978101", "Java Programming");
9         books.put("978102", "Python Basics");
10        books.put("978103", "Data Structures");
11
12        System.out.println("Book Found:");
13        System.out.println(books.get("978102"));
14
15        System.out.println("\nAll Books:");
16
17        for(Map.Entry<String,String> entry : books.entrySet()) {
18            System.out.println(entry.getKey() + " : " + entry.getValue());
19        }
20    }

```

Output

```

Book Found:
Python Basics

All Books:
978102 : Python Basics
978101 : Java Programming
978103 : Data Structures

=== Code Execution Successful ===

```

Q4: Employee Data Processing

(a) Advantages of Generics

1. Type safety
2. No explicit type casting
3. Better code reusability

(b) List Representation

ArrayList<Employee>

Employee

ID

Name

Salary

(c) Java Program

```
import java.util.*;
```

```
class Employee {
```

```
    int id;
```

```
    String name;
```

```
    double salary;
```

```
    Employee(int id, String name, double salary) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.salary = salary;
```

```
    }
```

```
}
```

```
public class EmployeeData {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<Employee> list = new ArrayList<>();
```

```
        list.add(new Employee(1,"Alice",60000));
```

```
        list.add(new Employee(2,"Bob",45000));
```

```
        list.add(new Employee(3,"Charlie",70000));
```

```
        Iterator<Employee> it = list.iterator();
```

```

        System.out.println("Salary > 50000");

        while(it.hasNext()) {
            Employee e = it.next();

            if(e.salary > 50000) {
                System.out.println(e.id+" "+e.name+" "+e.salary);
            }
        }
    }
}

```

Output

Q5: Online Voting System

(a) Three Suitable Collection Types

1. HashSet
2. HashMap
3. ArrayList

(b) Structure

HashSet

Voter1

Voter2

Voter3

HashMap

Candidate A -> 3

Candidate B -> 2

Candidate C -> 1

(c) Java Program

```
import java.util.*;
```

```
public class OnlineVoting {
```

```
    public static void main(String[] args) {
```

```
        HashSet<String> voters = new HashSet<>();
```

```
        HashMap<String, Integer> votes = new HashMap<>();
```

```
        String voter = "User1";
```

```
        String candidate = "Alice";
```

```
        if(voters.add(voter)) {
```

```
            votes.put(candidate, votes.getOrDefault(candidate,0)+1);
```

```
        } else {
```

```
            System.out.println("Duplicate Vote Not Allowed");
```

```
        }
```

```
        voter = "User2";
```

```
        candidate = "Bob";
```

```
        if(voters.add(voter)) {
```

```
            votes.put(candidate, votes.getOrDefault(candidate,0)+1);
```

```
        }
```

```

voter = "User1";

candidate = "Bob";

if(voters.add(voter)) {
    votes.put(candidate, votes.getDefault(candidate,0)+1);
} else {
    System.out.println("Duplicate Vote Not Allowed");
}

System.out.println("\nVoting Results:");

for(Map.Entry<String,Integer> entry : votes.entrySet()) {
    System.out.println(entry.getKey()+" : "+entry.getValue());
}
}
}

```

Output

The screenshot shows an online code editor with a file named 'OnlineVoting.java'. The code is as follows:

```

18 voter = "User2";
19 candidate = "Bob";
20
21 if(voters.add(voter)) {
22     votes.put(candidate, votes.getDefault(candidate,0)+1);
23 }
24
25 voter = "User1";
26 candidate = "Bob";
27
28 if(voters.add(voter)) {
29     votes.put(candidate, votes.getDefault(candidate,0)+1);
30 } else {
31     System.out.println("Duplicate Vote Not Allowed");
32 }
33
34 System.out.println("\nVoting Results:");
35
36 for(Map.Entry<String,Integer> entry : votes.entrySet()) {
37     System.out.println(entry.getKey()+" : "+entry.getValue());
38 }

```

The output panel on the right shows the following results:

```

Duplicate Vote Not Allowed

Voting Results:
Bob : 1
Alice : 1

=== Code Execution Successful ===

```